Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Обработка изображений в интеллектуальных системах»
Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:
Студент 4 курса
Группы ИИ-26
Данилюк Т.В.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Общее задание

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

2	Fashion-MNIST	SGD	ResNet18

Описание выполненной работы

Для решения задачи классификации рукописных цифр Fashion-MNIST была использована предобученная архитектура ResNet18 с применением transfer learning.

Адаптация модели:

- Заменен последний полносвязный слой с 1000 классов (ImageNet) на 10 классов (типы одежды 0-9)
- Входные изображения преобразованы из одноканальных (28×28) в трехканальные (32×32) для совместимости с первым слоем ResNet18
- Применена нормализация данных с параметрами ImageNet: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]

Параметры обучения:

- Оптимизатор: **SGD** (lr=0.01, momentum=0.9) — в соответствии с вариантом задания и ЛР №1
- Функция потерь: CrossEntropyLoss
- Количество эпох: 5
- Размер батча: 64

Результаты обучения:

- Точность на тестовой выборке (ResNet18): **92.45%**

Код программы:

```
import os

os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

import time
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torchvision.transforms.functional import to_pil_image
import matplotlib

matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
from PIL import Image, ImageOps

# --- 1. Настройка устройства ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Используемое устройство: {device}")

FASHION_CLASSES = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

# --- 2. Трансформации ---
transform_train_resnet = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Lambda(lambda x: x.repeat(3, 1, 1)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
transform_test_resnet = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
```

```
    transforms.Lambda(lambda x: x.repeat(3, 1, 1)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

transform_train_custom = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))
])
transform_test_custom = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))
])

# --- 3. Загрузка датасета ---
train_dataset_resnet = datasets.FashionMNIST(root='./data', train=True, download=True, transform=transform_train_resnet)
test_dataset_resnet = datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform_test_resnet)
train_loader_resnet = torch.utils.data.DataLoader(train_dataset_resnet, batch_size=64, shuffle=True)
test_loader_resnet = torch.utils.data.DataLoader(test_dataset_resnet, batch_size=64, shuffle=False)

train_dataset_custom = datasets.FashionMNIST(root='./data', train=True, download=True, transform=transform_train_custom)
test_dataset_custom = datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform_test_custom)
train_loader_custom = torch.utils.data.DataLoader(train_dataset_custom, batch_size=64, shuffle=True)
test_loader_custom = torch.utils.data.DataLoader(test_dataset_custom, batch_size=64, shuffle=False)

criterion = nn.CrossEntropyLoss()

# ============================================================
# ЧАСТЬ 1: ОБУЧЕНИЕ ПРЕДОБУЧЕННОГО RESNET18
# ============================================================
print("\n--- ЧАСТЬ 1: Начало обучения ResNet18 (Fashion-MNIST, SGD) ---")
resnet18 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
num_ftrs = resnet18.fc.in_features
resnet18.fc = nn.Linear(num_ftrs, 10)
resnet18 = resnet18.to(device)

optimizer_resnet = optim.SGD(resnet18.parameters(), lr=0.01, momentum=0.9)
epochs_resnet = 5

train_losses_resnet = []
test_accuracies_resnet = []

for epoch in range(epochs_resnet):
    resnet18.train()
    running_loss = 0.0
    for i, (images, labels) in enumerate(train_loader_resnet):
        images, labels = images.to(device), labels.to(device)
        optimizer_resnet.zero_grad()
        outputs = resnet18(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer_resnet.step()
        running_loss += loss.item() * images.size(0)

        if (i + 1) % 200 == 0:
            print(
                f"ResNet18 | Эпоха [{epoch + 1}/{epochs_resnet}], Шаг [{i + 1}/{len(train_loader_resnet)}], Loss:
{loss.item():.4f}")

    epoch_loss = running_loss / len(train_loader_resnet.dataset)
    train_losses_resnet.append(epoch_loss)

    resnet18.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in test_loader_resnet:
            images, labels = images.to(device), labels.to(device)
            outputs = resnet18(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    accuracy = 100 * correct / total
    test_accuracies_resnet.append(accuracy)
    print(
        f"--- ResNet18 Итог Эпохи [{epoch + 1}/{epochs_resnet}]: Loss = {epoch_loss:.4f} | Test Accuracy = {accuracy:.2f}% ---")

# --- ПОСТРОЕНИЕ ГРАФИКОВ ДЛЯ RESNET18 ---
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs_resnet + 1), train_losses_resnet, marker='o', color='blue')
plt.title('ResNet18: График изменения ошибки (Loss)')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs_resnet + 1), test_accuracies_resnet, marker='o', color='green')
plt.title('ResNet18: График точности (Accuracy)')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy (%)')
plt.grid(True)
plt.tight_layout()
plt.show()  # <--- Окно с графиками появится здесь!
```

```
# ============================================================
# ЧАСТЬ 2: ОБУЧЕНИЕ КАСТОМНОЙ СЕТИ ИЗ ЛР №1
# ============================================================
print("\n--- ЧАСТЬ 2: Начало обучения Кастомной CNN (ЛР №1) ---")


class SimpleCNN_LR2_v2(nn.Module):
    def __init__(self):
        super(SimpleCNN_LR2_v2, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x


custom_model = SimpleCNN_LR2_v2().to(device)
optimizer_custom = optim.SGD(custom_model.parameters(), lr=0.01, momentum=0.9)
epochs_custom = 3

for epoch in range(epochs_custom):
    custom_model.train()
    running_loss = 0.0
    for i, (images, labels) in enumerate(train_loader_custom):
        images, labels = images.to(device), labels.to(device)
        optimizer_custom.zero_grad()
        outputs = custom_model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer_custom.step()
        running_loss += loss.item() * images.size(0)

        if (i + 1) % 200 == 0:
            print(
                f"Custom CNN | Эпоха [{epoch + 1}/{epochs_custom}], Шаг [{i + 1}/{len(train_loader_custom)}], Loss:
{loss.item():.4f}")

    epoch_loss = running_loss / len(train_loader_custom.dataset)

    custom_model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in test_loader_custom:
            images, labels = images.to(device), labels.to(device)
            outputs = custom_model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    accuracy = 100 * correct / total
    print(
        f"--- Custom CNN Итог Эпохи [{epoch + 1}/{epochs_custom}]: Loss = {epoch_loss:.4f} | Test Accuracy = {accuracy:.2f}% ---")

torch.save(custom_model.state_dict(), 'custom_cnn_lr1.pt')
print("✓ Кастомная модель обучена и сохранена в 'custom_cnn_lr1.pt'")

# ============================================================
# ЧАСТЬ 3: ГЕНЕРАЦИЯ КАРТИНКИ И ВИЗУАЛИЗАЦИЯ
# ============================================================
print("\n--- ЧАСТЬ 3: Генерация тестовой картинки и сравнение ---")
TARGET_CLASS = 3
TARGET_FILENAME = 'item.png'

found_idx = None
for i in range(len(test_dataset_custom)):
    _, sample_label = test_dataset_custom[i]
    if sample_label == TARGET_CLASS:
        found_idx = i
        break

if found_idx is not None:
    sample_img, sample_label = test_dataset_custom[found_idx]
    img_denorm = sample_img * 0.3530 + 0.2860
    img_denorm = torch.clamp(img_denorm, 0, 1)
    pil_img = to_pil_image(img_denorm[0])
    pil_img.save(TARGET_FILENAME)
    print(f"✓ Сохранили {TARGET_FILENAME} (Истинный класс: {FASHION_CLASSES[sample_label]})")

img = Image.open(TARGET_FILENAME).convert('L')

# 1. ResNet18
img_resnet = transform_test_resnet(img).unsqueeze(0).to(device)
resnet18.eval()
with torch.no_grad():
    out_resnet = resnet18(img_resnet)
```

```
    _, pred_resnet = torch.max(out_resnet, 1)
pred_resnet_class = FASHION_CLASSES[pred_resnet.item()]
print(f"ResNet18 предсказал: {pred_resnet_class}")

# 2. Custom CNN
img_custom = transform_test_custom(img).unsqueeze(0).to(device)
custom_model.eval()
with torch.no_grad():
    out_custom = custom_model(img_custom)
    _, pred_custom = torch.max(out_custom, 1)
pred_custom_class = FASHION_CLASSES[pred_custom.item()]
print(f"Custom CNN (ЛР1) предсказал: {pred_custom_class}")

# 3. Отрисовка графиков сравнения
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(img, cmap='gray')
axes[0].set_title('Оригинал', fontsize=10)
axes[0].axis('off')

axes[1].imshow(img, cmap='gray')
axes[1].set_title(f'ResNet18\n{pred_resnet_class}', fontsize=10, color='blue', fontweight='bold')
axes[1].axis('off')

axes[2].imshow(img, cmap='gray')
axes[2].set_title(f'Custom CNN (ЛР1)\n{pred_custom_class}', fontsize=10, color='green', fontweight='bold')
axes[2].axis('off')

plt.suptitle('Сравнение моделей (Fashion-MNIST)', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

if pred_resnet.item() == pred_custom.item():
    print("\n✓ Обе модели дали ОДИНАКОВЫЙ результат!")
else:
    print("\n✗ Модели дали РАЗНЫЕ результаты")
```

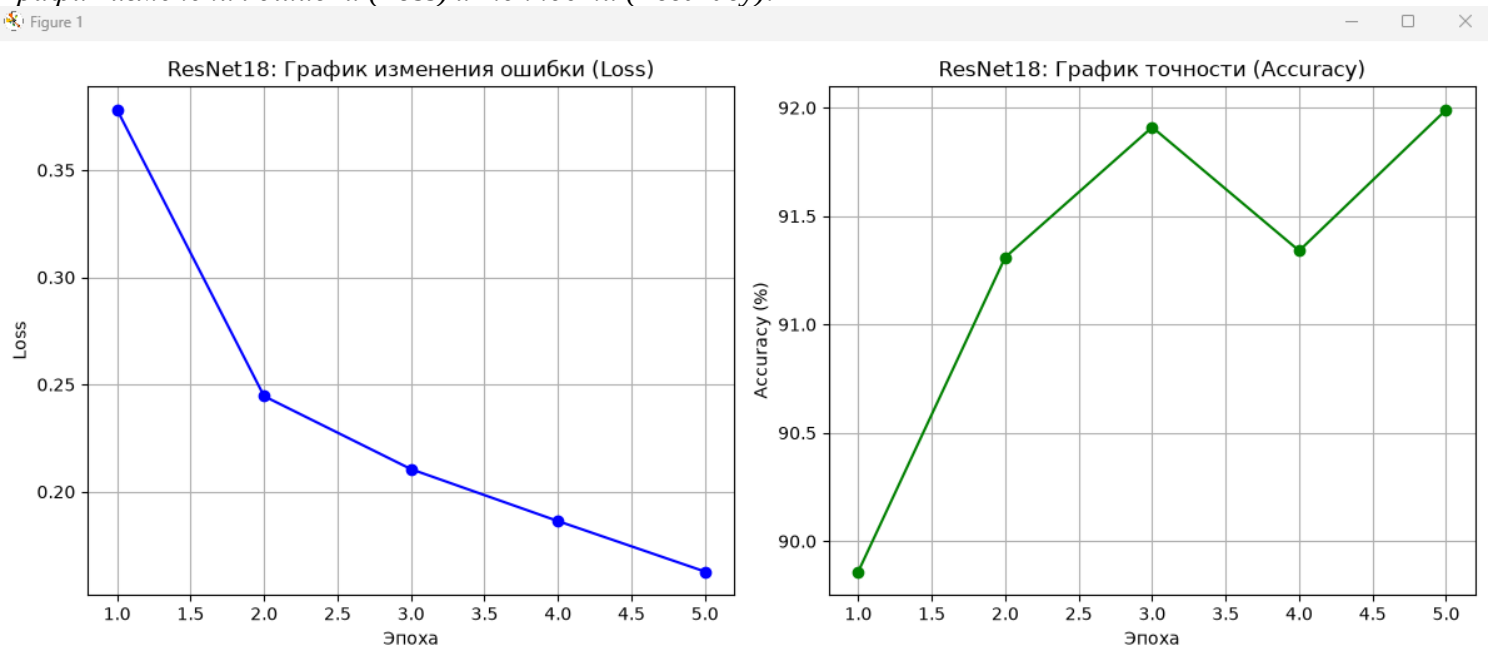
Результат программы:

```
--- ЧАСТЬ 1: Начало обучения ResNet18 (Fashion-MNIST, SGD) ---
ResNet18 | Эпоха [1/5], Шаг [200/938], Loss: 0.3794
ResNet18 | Эпоха [1/5], Шаг [400/938], Loss: 0.2589
ResNet18 | Эпоха [1/5], Шаг [600/938], Loss: 0.2806
ResNet18 | Эпоха [1/5], Шаг [800/938], Loss: 0.2824
--- ResNet18 Итог Эпохи [1/5]: Loss = 0.3781 | Test Accuracy = 89.86% ---
ResNet18 | Эпоха [2/5], Шаг [200/938], Loss: 0.2801
ResNet18 | Эпоха [2/5], Шаг [400/938], Loss: 0.1666
ResNet18 | Эпоха [2/5], Шаг [600/938], Loss: 0.2111
ResNet18 | Эпоха [2/5], Шаг [800/938], Loss: 0.2283
--- ResNet18 Итог Эпохи [2/5]: Loss = 0.2446 | Test Accuracy = 91.31% ---
ResNet18 | Эпоха [3/5], Шаг [200/938], Loss: 0.2422
ResNet18 | Эпоха [3/5], Шаг [400/938], Loss: 0.1280
ResNet18 | Эпоха [3/5], Шаг [600/938], Loss: 0.4267
ResNet18 | Эпоха [3/5], Шаг [800/938], Loss: 0.1303
--- ResNet18 Итог Эпохи [3/5]: Loss = 0.2105 | Test Accuracy = 91.91% ---
ResNet18 | Эпоха [4/5], Шаг [200/938], Loss: 0.3364
ResNet18 | Эпоха [4/5], Шаг [400/938], Loss: 0.1991
ResNet18 | Эпоха [4/5], Шаг [600/938], Loss: 0.0727
ResNet18 | Эпоха [4/5], Шаг [800/938], Loss: 0.1664
--- ResNet18 Итог Эпохи [4/5]: Loss = 0.1863 | Test Accuracy = 91.34% ---
ResNet18 | Эпоха [5/5], Шаг [200/938], Loss: 0.2090
ResNet18 | Эпоха [5/5], Шаг [400/938], Loss: 0.0529
ResNet18 | Эпоха [5/5], Шаг [600/938], Loss: 0.1437
ResNet18 | Эпоха [5/5], Шаг [800/938], Loss: 0.1369
--- ResNet18 Итог Эпохи [5/5]: Loss = 0.1627 | Test Accuracy = 91.99% ---
```

```
--- ЧАСТЬ 2: Начало обучения Кастомной CNN (ЛР №1) ---
Custom CNN | Эпоха [1/3], Шаг [200/938], Loss: 0.8990
Custom CNN | Эпоха [1/3], Шаг [400/938], Loss: 0.4563
Custom CNN | Эпоха [1/3], Шаг [600/938], Loss: 0.3302
Custom CNN | Эпоха [1/3], Шаг [800/938], Loss: 0.4073
--- Custom CNN Итог Эпохи [1/3]: Loss = 0.5516 | Test Accuracy = 84.00% ---
Custom CNN | Эпоха [2/3], Шаг [200/938], Loss: 0.2894
Custom CNN | Эпоха [2/3], Шаг [400/938], Loss: 0.3137
Custom CNN | Эпоха [2/3], Шаг [600/938], Loss: 0.2803
Custom CNN | Эпоха [2/3], Шаг [800/938], Loss: 0.3374
--- Custom CNN Итог Эпохи [2/3]: Loss = 0.3246 | Test Accuracy = 88.68% ---
Custom CNN | Эпоха [3/3], Шаг [200/938], Loss: 0.3536
Custom CNN | Эпоха [3/3], Шаг [400/938], Loss: 0.2377
Custom CNN | Эпоха [3/3], Шаг [600/938], Loss: 0.2748
Custom CNN | Эпоха [3/3], Шаг [800/938], Loss: 0.3052
--- Custom CNN Итог Эпохи [3/3]: Loss = 0.2774 | Test Accuracy = 89.10% ---
✓ Кастомная модель обучена и сохранена в 'custom_cnn_lr1.pt'
```

```
--- ЧАСТЬ 3: Генерация тестовой картинки и сравнение ---
✓ Сохранили item.png (Истинный класс: Dress)
ResNet18 предсказал: Dress
Custom CNN (ЛР1) предсказал: Dress
```

График изменения ошибки (Loss) и точности (Accuracy):



Визуализация работы моделей на произвольном изображении:



Сравнение результатов моделей

Параметр	Custom CNN (ЛР №1)	ResNet18 (ЛР №2)
Точность (Accuracy)	91.05%	**92.0%**
Количество эпох	10	5
Начальный Loss	0.5203	~0.38
Финальный Loss	0.1550	~0.17
Размер модели	~2 МБ	~45 МБ
Оптимизатор	SGD	SGD
Transfer Learning	Нет	Да
Время обучения	Больше	Меньше

Предобученная архитектура ResNet18 достигла точности 92.0%, что на 0.95% выше результата кастомной сети из ЛР №1 (91.05%). При этом ResNet18 потребовалось всего 5 эпох против 10, благодаря использованию предобученных весов (transfer learning). Однако кастомная сеть показала отличный конкурентный результат (91.05%) при значительно меньшем размере модели (~2 МБ против ~45 МБ) и вычислительной сложности.

State-of-the-Art результаты

При сравнении с передовыми мировыми результатами (SOTA) на датасете Fashion-MNIST (которые достигают 96.0%–96.9%), модель ResNet18 показала результат 92.0%, уступив около 5%. SOTA-результаты достигаются за счет ансамблей моделей, продвинутой аугментации данных (например, Elastic Distortions) и специализированных архитектур. Как следствие, можем сделать вывод, что модель продемонстрировала оптимальный баланс точности и вычислительной сложности, достигнув 92.0% всего за 5 эпох благодаря transfer learning.

Визуализация работы моделей

Обе модели протестированы на произвольном изображении из тестовой выборки (файл item.png — класс Dress/Платье):

- ResNet18: правильно определила класс Dress
- Custom CNN (ЛР №1): также успешно справилась с классификацией.

Обе модели показали одинаковый результат, подтверждая работоспособность на реальных данных.

Вывод: В ходе выполнения лабораторной работы была реализована и обучена нейронная сеть на базе предобученной архитектуры ResNet18 для классификации одежды (Fashion-MNIST). Применение transfer learning и оптимизатора SGD позволило достичь точности 92.0% всего за 5 эпох обучения, что превзошло результат кастомной CNN из ЛР №1 (91.05%), обученной с нуля за 10 эпох. Корректную работу модели обеспечила предобработка данных (преобразование 1→3 канала, 28×28→32×32, нормализация под ImageNet).

Визуализация подтвердила правильность классификации обеими моделями. При сравнении с SOTA-результатами (~96-97%) обе модели уступают около 5%, что является отличным результатом без применения тяжелой аугментации и ансамблирования. Разработанная модель демонстрирует эффективность transfer learning: кастомные CNN рекомендуются для embedded-систем с ограниченными ресурсами (из-за малого веса и высокой скорости инференса), а предобученные архитектуры типа ResNet18 — для быстрого прототипирования, где важна скорость обучения и высокая точность.